1. What is React? (The UI Library)

React is an open-source JavaScript library developed by Meta (Facebook) used for building User Interfaces (UIs), specifically interactive web applications.

- **Component-Based Architecture:** In traditional HTML/CSS/JS, you write large monolithic HTML files. In React, you break down the UI into small, reusable, independent pieces called **Components** (e.g., a `Navbar`, a `ProductCard`, a `CartButton`).
- **JSX (JavaScript XML):** React lets you write HTML code directly inside JavaScript files using JSX. It looks like HTML, but it's actually JavaScript under the hood, allowing you to embed dynamic variables and logic straight into your markup using curly braces `{}`.
- **State and Props:**
    - **Props (Properties):** Read-only data passed down from a parent component to a child component (like arguments to a function).
    - **State:** Internal, mutable data belonging to a component. When state changes, React automatically re-renders the component to reflect the new data on the screen.

2. React SPA (Vite/Create React App) + Separate Backend vs. Next.js

Before Next.js existed, the standard modern web architecture looked like a **Single Page Application (SPA)** decoupled from a backend API. Let's compare the two approaches:

Approach A: Separate Frontend (React SPA) + Separate Backend (Node.js/Express)

1. **How it works:**
    - Your frontend code (React) lives in one repository/folder and runs on a development server (like `http://localhost:5173` via Vite).
    - Your backend code (Node.js/Express, which you are already familiar with) lives in another repository or folder and runs on a different port (like `http://localhost:5000`).
2. **The Flow:**
    - When a user visits your app, the browser downloads an empty HTML shell and a massive bundle of JavaScript.
    - The browser executes the JavaScript to render the UI on the client-side (Client-Side Rendering or CSR).

    - If the React app needs data (like products), it makes an HTTP fetch request (e.g., `axios.get('http://localhost:5000/api/products')`) across the network to your Express server.
3. **Pros & Cons:**
    - *Pros:* Clear separation of concerns; backend can serve multiple clients (web, mobile apps).
    - *Cons:* Slower initial page load because the browser has to do all the heavy lifting of rendering. SEO (Search Engine Optimization) can be challenging because search engine crawlers see an empty HTML file initially. Managing CORS, dual servers, and environment variables across two codebases adds overhead.

Approach B: Next.js (The Full-Stack Framework)

Next.js is a framework built *on top* of React by Vercel. It solves the pain points of a pure React SPA by providing a **Full-Stack Meta-Framework**.

1. **How it works:**
    - Next.js combines both frontend UI and backend capabilities into a **single project/codebase**.
    - By default, Next.js uses the **App Router** (introduced in modern Next.js versions), where pages and layouts are organized inside an `app/` directory.
2. **Rendering Flexibility (Server-Side Rendering & Client Components):**
    - **Server Components (RSC):** By default, components in Next.js render on the server. This means database queries, API secrets, and heavy computations happen securely on the server before the HTML is even sent to the browser. This results in lightning-fast initial page loads and excellent SEO.
    - **Client Components:** When you need interactivity (like `useState`, `useEffect`, or event listeners like `onClick`), you explicitly mark a component with `'use client'`, telling Next.js to render it on the client side just like a traditional React app.
3. **Built-in Backend Capabilities (Route Handlers):**
    - You don't need a separate Express server. Next.js allows you to build backend API endpoints right inside your project using special files named `route.ts` (e.g., `app/api/products/route.ts`).

What I Accomplished & Learned

1. **Full-Stack Architecture & Supabase Integration**
    - Migrated away from static local mock data (`src/data/products.ts`) to a live cloud database backend using **Supabase**.

    - Implemented dynamic database fetching inside a Next.js Client Component using `useEffect` and the Supabase client (`supabase.from("products").select("*")`).
2. **TypeScript & Type Safety**
    - Defined strict data contracts and interfaces (`Product` and `CartItem`) in `src/types/product.ts` to ensure type safety across product listings, stock tracking, and shopping cart quantities.
3. **Interactive E-Commerce State & Cart Logic**
    - Built robust shopping cart functionality supporting:
        - Adding items with stock limit validation.
        - Incrementing/decrementing quantities dynamically.
        - Removing items from the cart.
        - Real-time total calculation and out-of-state badges.
4. **Version Control & GitHub Workflow**
    - Initialized and organized the project repository structure locally.
    - Created a dedicated feature branch (`feature/product-list-and-cart`).
    - Linked the local repository to a newly created remote repository on GitHub and pushed the codebase successfully.

🛠️ Roadblocks We Faced & How We Fixed Them

1. The "Missing Products / Showing Old Data" Issue

- **The Problem:** Initially, the app was only rendering 4 old mock items instead of the 11 records stored in Supabase, or caching previous builds.
- **How We Fixed It:**
    - Checked the Supabase Table Editor to visually verify that all 11 records were successfully inserted.
    - Cleared stale Next.js cache and restarted the development server (`npm run dev`) so it could freshly ingest the environment variables and clean data flows.

2. TypeScript Module Resolution Errors (`ts(2307)`)

- **The Problem:** VS Code flagged breaking TypeScript compilation errors (`Cannot find module '@/types/product'`) because the types file hadn't been created yet, and legacy files (`src/data/products.ts`) were still trying to import from non-existent paths.
- **How We Fixed It:**
    - Created the missing `src/types/product.ts` file containing the clean `Product` and `CartItem` interfaces.

    - Cleaned up and completely decoupled `src/app/page.tsx` from the legacy mock data imports, replacing them entirely with live Supabase database calls.

3. Capitalized Database Columns & Missing Prices (`$0.00`)

- **The Problem:** Supabase tables created via the dashboard UI occasionally capitalize column headers (`Price`, `Image`, `Stock`), causing the application to read `undefined` values and display `$0.00`.
- **How We Fixed It:**
    - Updated our data-fetching normalization logic in `page.tsx` to safely check for *both* lowercase and uppercase field variants (`item.price || item.Price`, `item.image || item.Image`, etc.), ensuring accurate data mapping regardless of column casing.

4. Unsplash Image Hotlinking & "Image Error" Placeholders

- **The Problem:** Direct URLs pointing to external image hosts like Unsplash threw hotlinking errors or failed to load inside standard `<img>` tags, resulting in broken image cards.
- **How We Fixed It:**
    - Integrated reliable, dynamic placeholder image generation (`[https://picsum.photos/seed/](https://picsum.photos/seed/)...`) inside our product normalization loop to guarantee clean, high-speed image rendering across all catalog items.